

ITERATORS & HIGHER-ORDER PROGRAMMING

Presented By Group 20

Member

1. นาย ณัฐชนน รักวงษ์ — Concept + Short Code Illustration
2. นางสาว ณัฐชาพร เสตะวีระ — Detailed Code + Live Demo
3. นาย ณัฐดนัย ศรีไวย — Rust vs Other Language + PPL Analysis
4. นาย พรสวกร สมใจ — Exercises + Common Mistakes + Challenge



Agenda - โครงสร้างการนำเสนอ (รวม ~20 นาที)

แต่ละคนนำเสนอประมาณ 5 นาที ต่อเนื่องกันตามลำดับ



Member 1

**Concept + Short Code
Illustration**

แนวคิดหลักของหัวข้อ พร้อมโค้ดตัวอย่างสั้นๆ

~5 นาที



Member 2

**Detailed Code + Live
Demo**

อธิบายโค้ดเชิงลึกและสาธิตการทำงานจริง

~5 นาที



Member 3

**Rust vs Other Language
+ PPL Analysis**

เปรียบเทียบภาษา และวิเคราะห์เชิง
Programming Language

~5 นาที



Member 4

**Exercises + Common
Mistakes + Challenge**

ข้อผิดพลาดที่พบบ่อย และโจทย์ท้าทายผู้
ฟัง

~5 นาที

Concepts

- Iterator: ประมวลผลข้อมูลที่ละตัวแบบ Lazy Evaluation ช่วยประหยัดหน่วยความจำ
 - Higher-Order Function: ฟังก์ชันที่รับ/คืนฟังก์ชันอื่นเป็น Argument/Result
 - เมธอดหลัก: map, filter, fold ใช้ประมวลผลข้อมูลโดยไม่ต้องใช้ for loop
- iter()
 - map()
 - filter()
 - fold()
 - collect()
 - lazy evaluation

iter()

สร้างตัววนซ้ำ (Iterator) โดยขออ่านข้อมูล โดยส่งคืนค่าเป็น Reference

Output

```
fn main() {  
    let numbers = vec![1, 2, 3];  
  
    for number in numbers.iter() {  
        println!("{}", number);  
    }  
}
```

```
key concept 1: iter()  
1  
2  
3
```

map()

แปลงข้อมูลทุกตัวใน Iterator โดยส่งคืนค่าเป็น Iterator ตัวใหม่ ในแต่ละรอบ number จะเป็น Reference (&i32) ที่ชี้ไปยังข้อมูลเดิม ดังนั้น ข้อมูลใน numbers ยังคงสามารถใช้งานต่อได้หลังจากวนลูปเสร็จ

Output

```
fn main() {  
    let numbers = vec![1, 2, 3];  
  
    let doubled = numbers.iter().map(|number| number * 2);  
  
    for number in doubled {  
        println!("{}", number);  
    }  
}
```

key concept 2: map()

2
4
6

filter()

คัดกรองข้อมูลตามเงื่อนไข โดย Closure ต้องคืนค่าเป็น true หรือ false

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    let even_numbers = numbers  
        .iter()  
        .filter(|number| **number % 2 == 0);  
  
    for number in even_numbers {  
        println!("{}", number);  
    }  
}
```

Output

```
key concept 3: filter()  
2  
4
```

fold()

ยุบรวมข้อมูลทั้งหมดใน Iterator ให้เหลือค่าเดียว โดยคั่นค่าผลลัพธ์
สุดท้ายชนิดเดียว

```
fn main() {  
    let numbers = vec![1, 2, 3, 4];  
  
    let sum = numbers  
        .iter()  
        .fold(0, |total, number| total + number);  
  
    println!("{}", sum);  
}
```

Output

```
key concept 4: fold()  
10
```

collect()

รวบรวมข้อมูลจาก Iterator กลับไปเป็น Collection คัดค้านค่าเป็น collection ใหม่ เช่น Vec, HashMap

```
fn main() {  
    let numbers = vec![1, 2, 3];  
  
    let doubled: Vec<i32> = numbers  
        .iter()  
        .map(|number| number * 2)  
        .collect();  
  
    println!("{:?}", doubled);  
}
```

Output

```
key concept 5: collect()  
[2, 4, 6]
```

lazy evaluation

lazy evaluation จะคำนวณผลลัพธ์ต่อเมื่อมีการเรียกให้แสดงผลลัพธ์เท่านั้น
ช่วยลดการคำนวณที่ไม่จำเป็น

```
fn main() {  
    let numbers = vec![1, 2, 3];  
  
    let doubled: Vec<i32> = numbers  
        .iter()  
        .map(|number| number * 2)  
        .collect();  
  
    println!("{:?}", doubled);  
}
```

explain

ที่ `.map()` ยังไม่คำนวณ
คำนวณที่ `.collect()` เนื่องจาก
ต้องรวบรวมข้อมูลจาก Iterator
กลับไปเป็น Collection



```
fn main() {
    let scores = vec![35, 50, 68, 72, 90, 45, 80];

    println!("Original Scores: {:?}", scores);

    let scholarship_students = scores // ยังไม่คำนวณ สร้าง pipeline ไว้ก่อน
        .iter()
        .filter(|score| **score >= 60)
        .map(|score| {
            println!("Adding bonus to: {}", score);
            score + 5
        });

    //จุดที่เริ่มประมวลผลจริง
    let final_scores: Vec<i32> = scholarship_students.collect();

    println!("\n===== Scholarship Award Results =====");

    for score in &final_scores {
        println!("Student recieved scholarship | Final score: {}", score);
    }

    println!("\nTotal Scholarship Students: {}", final_scores.len());

    println!("===== Congratuations ! =====");
}
```

ระบบคัดเลือกนักศึกษารับทุน (Scholarship system)

Demo Checklist

□ Step 1: แสดงคะแนนเริ่มต้นของนักศึกษาทั้งหมด

ขั้นแรก โปรแกรมจะแสดงคะแนนเริ่มต้นของนักศึกษาทั้งหมดก่อน ซึ่งในที่นี้มีคะแนน 35, 50, 68, 72, 90, 45 และ 80

□ Step 2: คัดเลือกนักศึกษาที่ผ่านเกณฑ์

จากนั้นโปรแกรมจะใช้ filter() เพื่อคัดเลือกเฉพาะนักศึกษาที่มีคะแนนตั้งแต่ 60 คะแนนขึ้นไป เพราะเป็นเกณฑ์ที่กำหนดสำหรับทุน

□ Step 3: เพิ่ม Bonus

สำหรับนักศึกษาที่ผ่านเกณฑ์ โปรแกรมจะใช้ map() เพื่อเพิ่มคะแนน Bonus อีก 5 คะแนน โดยจะเห็นข้อความแสดงคะแนนก่อนเพิ่ม Bonus ออกมาด้วย

□ Step 4: แสดงผลลัพธ์สุดท้าย

สุดท้าย โปรแกรมจะ collect() ผลลัพธ์ทั้งหมดมาเก็บใน final_scores แล้วแสดงคะแนนสุดท้ายของนักศึกษาที่ได้รับทุน รวมถึงจำนวนผู้ได้รับทุนทั้งหมด

สิ่งที่ต้องระวังระหว่าง Demo

- ตรวจสอบว่าเฉพาะคะแนน ≥ 60 เท่านั้นที่ได้รับ Bonus
- ตรวจสอบคะแนนสุดท้ายและจำนวนผู้ได้รับทุน



```
fn main() {  
    let scores = vec![35, 50, 68, 72, 90, 45, 80];  
  
    println!("Original Scores: {:?}", scores);  
  
    let scholarship_students = scores // ยังไม่คำนวณ สร้าง pipeline ไว้ก่อน  
        .iter()  
        .filter(|score| **score >= 60)  
        .map(|score| {  
            println!("Adding bonus to: {}", score);  
            score + 5  
        });  
  
    //จุดที่เริ่มประมวลผลจริง  
    let final_scores: Vec<i32> = scholarship_students.collect();  
  
    println!("\n===== Scholarship Award Results =====");  
  
    for score in &final_scores {  
        println!("Student recieved scholarship | Final score: {}", score);  
    }  
  
    println!("\nTotal Scholarship Students: {}", final_scores.len());  
  
    println!("===== Congratuations ! =====");  
}
```

ระบบคัดเลือกนักศึกษารับทุน (Scholarship system)

Output

```
Original Scores: [35, 50, 68, 72, 90, 45, 80]  
Adding bonus to: 68  
Adding bonus to: 72  
Adding bonus to: 90  
Adding bonus to: 80  
  
===== Scholarship Award Results =====  
Student recieved scholarship | Final score: 73  
Student recieved scholarship | Final score: 77  
Student recieved scholarship | Final score: 95  
Student recieved scholarship | Final score: 85  
  
Total Scholarship Students: 4  
===== Congratuations ! =====
```

```
fn main() {
    let menus = vec![
        ("Burger", 150),
        ("French Fries", 80),
        ("Pizza", 250),
        ("Coke", 50),
        ("Steak", 300),
    ];

    let total = menus
        .iter()
        // เลือกเฉพาะเมนูที่ราคาไม่เกิน 200 บาท
        .filter(|(&_, price)| *price <= 200)
        // ลดราคา 10%
        .map(|(&_, price)| *price - (*price / 10))
        // รวมราคาทั้งหมด
        .fold(0, |acc, price| acc + price);

    println!("=====");
    println!("      FOOD ORDER SUMMARY");
    println!("=====");

    println!("Price Limit : 200 Baht");
    println!("Discount      : 10%");
    println!("-----");

    for (name, price) in &menus {
        if *price <= 200 {
            let final_price = *price - (*price / 10);

            println!(
                "{} | Original: {} Baht | Final: {} Baht | Selected",
                name, price, final_price
            );
        } else {
            println!("{}", name, price);
        }
    }

    println!("-----");
    println!("Total Payment: {} Baht", total);
    println!("=====");
}
```

ระบบสั่งอาหาร (Food Order System)

Demo Checklist

□ Step 1: แสดงรายการอาหารและราคาเริ่มต้น

ขั้นแรก โปรแกรมจะแสดงรายการอาหารพร้อมราคาตั้งต้นของแต่ละเมนู

□ Step 2: คัดเลือกเมนูที่ราคา ≤ 200 บาท

จากนั้นใช้ filter() เพื่อเลือกเฉพาะเมนูที่มีราคาไม่เกิน 200 บาท ส่วนเมนูที่เกิน 200 บาทจะไม่ถูกเลือก

□ Step 3: ลดราคา 10% สำหรับเมนูที่ถูกเลือก

เมนูที่ผ่านเงื่อนไขจะถูกนำมาคำนวณส่วนลด 10% ด้วย map() ทำให้ได้ราคาหลังหักส่วนลด เช่น Burger: 150 $\rightarrow$ 135 บาท

□ Step 4: แสดงรายการที่เลือกและ ยอดชำระทั้งหมด

สุดท้ายใช้ fold() เพื่อรวมราคาหลังส่วนลดของเมนูที่เลือกทั้งหมด แล้วแสดงยอดชำระรวม

สิ่งที่ต้องระวังระหว่าง Demo

- เมนูราคา เกิน 200 บาทต้องไม่ถูกเลือก
- ตรวจสอบว่าส่วนลด 10% ถูกคำนวณจากราคาต้นฉบับ
- ตรวจสอบยอดรวมหลังหักส่วนลดแล้ว

```
fn main() {
    let menus = vec![
        ("Burger", 150),
        ("French Fries", 80),
        ("Pizza", 250),
        ("Coke", 50),
        ("Steak", 300),
    ];

    let total = menus
        .iter()
        // เลือกเฉพาะเมนูที่ราคาไม่เกิน 200 บาท
        .filter(|(&_, price)| *price <= 200)
        // ลดราคา 10%
        .map(|(&_, price)| *price - (*price / 10))
        // รวมราคาทั้งหมด
        .fold(0, |acc, price| acc + price);

    println!("=====");
    println!("      FOOD ORDER SUMMARY");
    println!("=====");

    println!("Price Limit : 200 Baht");
    println!("Discount    : 10%");
    println!("-----");

    for (name, price) in &menus {
        if *price <= 200 {
            let final_price = *price - (*price / 10);

            println!(
                "{} | Original: {} Baht | Final: {} Baht | Selected",
                name, price, final_price
            );
        } else {
            println!("{}", name, price);
        }
    }

    println!("-----");
    println!("Total Payment: {} Baht", total);
    println!("=====");
}
```

ระบบสั่งอาหาร (Food Order System)

Output

```
=====
      FOOD ORDER SUMMARY
=====
Price Limit : 200 Baht
Discount    : 10%
-----
Burger | Original: 150 Baht | Final: 135 Baht | Selected
French Fries | Original: 80 Baht | Final: 72 Baht | Selected
Pizza | 250 Baht | Not Selected
Coke | Original: 50 Baht | Final: 45 Baht | Selected
Steak | 300 Baht | Not Selected
-----
Total Payment: 252 Baht
=====
```

Rust vs Other Language

Aspect	Rust	Python	Java	C
Syntax	Iterator chain .iter().filter().map().sum() ส่งฟังก์ชันหรือ closure เข้า adaptor	map(f, filter(g, data)) ใช้ lambda หรือ generator expression	Stream API .stream().filter().mapToInt().sum() ส่ง lambda เข้า operation	ไม่มี syntax พิเศษ เขียนลูป for เองและส่ง function pointer int (*f)(int)
Semantics / Behavior	Iterator เป็น Lazy ทำงานเมื่อมี consumer และ next() คืน Option	map, filter, generator เป็น Lazy ผ่าน Iterator Protocol (__next__) แต่ list comprehension คำนวณทันที	Stream เป็น Lazy ทำงานเมื่อเจอ terminal operation และใช้ซ้ำไม่ได้หลังถูกใช้แล้ว	ไม่มี Lazy Evaluation ลูปคำนวณทันทีตามที่เขียน
Type System	Static + strong typing ใช้ Iterator<Item = T> และ Trait Fn ตรวจสอบตอน Compile	Dynamic + strong typing ตรวจสอบ type ของ argument ใน lambda ตอน Runtime	Static + strong typing ใช้ Stream<T> และ Functional Interface เช่น Predicate, Function	Static typing function pointer ระบุ signature ได้ แต่ compiler ตรวจสอบได้จำกัด (cast ผิดชนิดได้)
Memory Management	Ownership, Borrowing, Lifetime ไม่มี Garbage Collector	Reference Counting ร่วมกับ Garbage Collector	Garbage Collector และ Stream สร้าง Object ภายในระหว่าง pipeline	Programmer จัดการเอง (malloc/free) ไม่มี Garbage Collector
Safety	Compiler ตรวจสอบการยืมและการ move ของ iterator ก่อน compile ป้องกัน Iterator Invalidation	แก้ list ขณะวนลูปได้ ผลลัพธ์อาจผิดปกติโดยไม่มี error	แก้ collection ขณะวนลูปอาจเกิด ConcurrentModificationException ตอน Runtime	ไม่ตรวจสอบขอบเขต array เข้าถึงเกิน index จะเกิด Undefined Behavior

PPL Perspective

Part 3 - Member 3

Syntax

การประมวลผลข้อมูลใน Rust แบบ Iterator เขียนเป็นลูกโซ่ 3 ส่วน คือ Source → Adaptor → Consumer โดย Adaptor เช่น map และ filter เป็น Higher-Order Function ที่รับ "ฟังก์ชัน" เข้าไปกำหนดพฤติกรรม

- Source: .iter(), .into_iter(), 1..=5 สร้าง iterator จากข้อมูล
- Adaptor: .map(), .filter(), .take(), .enumerate(), .zip() คัด iterator ตัวใหม่
- Consumer: .collect(), .sum(), .fold(), .count() ดึงค่าออกมาเป็นผลลัพธ์
- for x in iterator { ... } วนลูปผ่าน iterator
- fn func<F: Fn(i32) -> i32>(f: F) ฟังก์ชันที่รับฟังก์ชันเป็น parameter (ส่งได้ทั้งฟังก์ชันชื่อและ closure |x| x + 1)

```
fn is_even(x: &i32) -> bool {
    x % 2 == 0
}

fn square(x: i32) -> i32 {
    x * x
}

fn main() {
    let numbers = vec![1, 2, 3, 4, 5, 6];

    let result: Vec<i32> = numbers
        .iter()           // Source
        .copied()         // Adaptor: copy numbers แยกกับ numbers ownership ใหม่
        .filter(is_even)
        .map(square)
        .collect();       // Consumer

    println!("{:?}", result);
}
```

[4, 16, 36]

PPL Perspective

Part 3 - Member 3

Semantics

- Iterator ทุกตัวทำงานผ่าน method next() ที่คืน Option โดย Some(value) คือยังมีข้อมูล และ None คือหมดแล้ว
- loop for ก็คือการเรียก next() ซ้ำจนได้ None
- Adaptor เป็น Lazy Evaluation คือ ไม่คำนวณจนกว่าจะมี Consumer มาดึงค่า
- Short-Circuit จะหยุดเมื่อเจอสิ่งที่ต้องการแล้ว take_while, any จึงใช้กับเลขไม่รู้จบได้ เช่น (1..)

```
let v: Vec<i32> = [1, 2, 3, 10, 4, 5]
  .iter()
  .copied()
  .take_while(|&x| x < 5)
  .collect();
```

[1, 2, 3]

```
let has_even = [1, 3, 4, 7].iter().any(|&x| x % 2 == 0); // true, หยุดที่ 4
```

```
fn main() {
  let numbers = vec![1, 2, 3, 4, 5];

  let first_two: Vec<i32> = numbers
    .iter()
    .map(|x| {
      println!("map {}", x);
      x * 2
    })
    .take(2)
    .collect();

  println!("{:?}", first_two);
}
```

map 1
map 2
[2, 4]

PPL Perspective

Part 3 - Member 3

Type System

- `Iterator<Item = T>` คือ iterator ที่ส่งค่าชนิด `T` ออกมาทีละตัว (`Item` เป็น Associated Type)
- Adaptor แต่ละตัวคืน type ใหม่ที่ห่อตัวเดิมไว้ เช่น `Map<I, F>` และ `Filter<I, P>` ซึ่ง compiler รู้ตั้งแต่ตอน Compile
- Parameter ที่เป็นฟังก์ชันระบุด้วย Trait Bound เช่น `fn func<F: Fn(i32) -> i32>(f: F, value: i32) -> i32` หรือใช้ function pointer `fn func(f: fn(i32) -> i32, value: i32) -> i32`
- `collect()` ต้องรู้ชนิดปลายทาง เช่น `Vec<i32>` เพราะ `collect` เป็น `Vec`, `HashSet` หรือ `String` ได้

```
let v = vec![1, 2, 3, 4];
```

```
let a = v.iter();
```

```
// Iter<i32>
```

```
let b = a.filter(|x| **x > 1);
```

```
// Filter<Iter<i32>, closure1>
```

```
let c = b.map(|x| x * 10);
```

```
// Map<Filter<Iter<i32>, closure1>, closure2>
```

```
fn func<I: Iterator<Item = i32>, F: Fn(i32) -> i32>(items: I, f: F) -> Vec<i32> {  
    items.map(f).collect()  
}
```

PPL Perspective

Part 3 - Member 3

Memory / Resource Management

Iterator ใช้ระบบ Ownership และ Borrowing โดยวิธีสร้าง iterator ที่ต่างกันมีผลต่อ collection เดิมต่างกัน

- `iter()`: ยืมข้อมูลมาอ่าน (&T) collection เดิมยังใช้ได้
- `iter_mut()`: ยืมข้อมูลมาแก้ไข (&mut T)
- `into_iter()`: ย้าย ownership เข้า iterator (T) collection เดิมใช้ต่อไม่ได้
- Iterator Chain ไม่สร้าง collection ชั่วคราวระหว่างขั้นตอน จึงไม่เปลือง Heap เพิ่ม

```
fn main() {  
    let mut scores = vec![50, 60, 70];  
  
    let total: i32 = scores.iter().sum();           // ยืมอ่าน  
    println!("{}", total, scores);  
  
    for s in scores.iter_mut() {                   // ยืมแก้ไข  
        *s += 10;  
    }  
    println!("{}", scores);  
  
    let doubled: Vec<i32> = scores.into_iter().map(|s| s * 2).collect(); // ย้าย ownership  
    // println!("{}", scores); // Error: scores ถูก move แล้ว  
    println!("{}", doubled);  
}
```

```
180 [50, 60, 70]  
[60, 70, 80]  
[120, 140, 160]
```

PPL Perspective

Part 3 - Member 3

Abstraction / Other PPL Concepts

- Iterator เป็น Abstraction ของ "การ loop ข้อมูลจากตัวแรกจนถึงตัวสุดท้าย"
- Scope: ตัวแปรของ iterator และ closure อยู่ภายใต้ Lexical Scope
- Binding: รับฟังก์ชันผ่าน Generic (F: Fn) ผู้คอมไพล์ (Static Dispatch) ส่วนกรณีที่ไม่รู้ชนิดจริงจะรู้ตอนรันโปรแกรม dyn ผู้คอมไพล์ Runtime (Dynamic Dispatch)
- Paradigm: Rust เป็น Multi-paradigm โดย Iterator และ Higher-Order Function รองรับ Functional Programming (First-class Function) แต่ไม่ใช่ภาษา Functional แบบ pure เพราะยังมี Side Effect (ฟังก์ชันทำนอกเหนือจากการคืนค่า เช่น พิมพ์ออกจอ เขียนไฟล์ แก้ตัวแปรภายนอก)

```
trait Iterator {  
    type Item;  
    fn next(&mut self) -> Option<Self::Item>;  
}
```

```
struct Countdown(u32);  
  
impl Iterator for Countdown {  
    type Item = u32;  
  
    fn next(&mut self) -> Option<u32> {  
        if self.0 == 0 {  
            None  
        } else {  
            self.0 -= 1;  
            Some(self.0 + 1)  
        }  
    }  
}  
  
fn main() {  
    let total: u32 = Countdown(5).filter(|x| x % 2 == 1).sum();  
    println!("{}", total);  
}
```

ppl Perspective

Part 3 - Member 3

Why Rust?

- Memory Safety: Borrow Checker ตรวจสอบการยืมและการ move ของ iterator ป้องกันการใช้ข้อมูลที่ถูก move ไปแล้วหรือถูกแก้ไขขณะวนลูป
- Reliability: next() คืน Option จึงต้องจัดการกรณีข้อมูลหมดอย่างชัดเจน และ type ของ Adaptor ทุกตัวถูกตรวจสอบตอน Compile
- Performance: Iterator Chain เป็น Zero-Cost Abstraction เพราะ Lazy ไม่สร้าง collection ชั่วคราว และ compiler inline ฟังก์ชันที่ส่งผ่าน Generic ได้
- No Garbage Collector: จัดการทรัพยากรด้วย Ownership และจัดการค่าตาม scope

Common Mistakes | ข้อผิดพลาดที่พบบ่อย

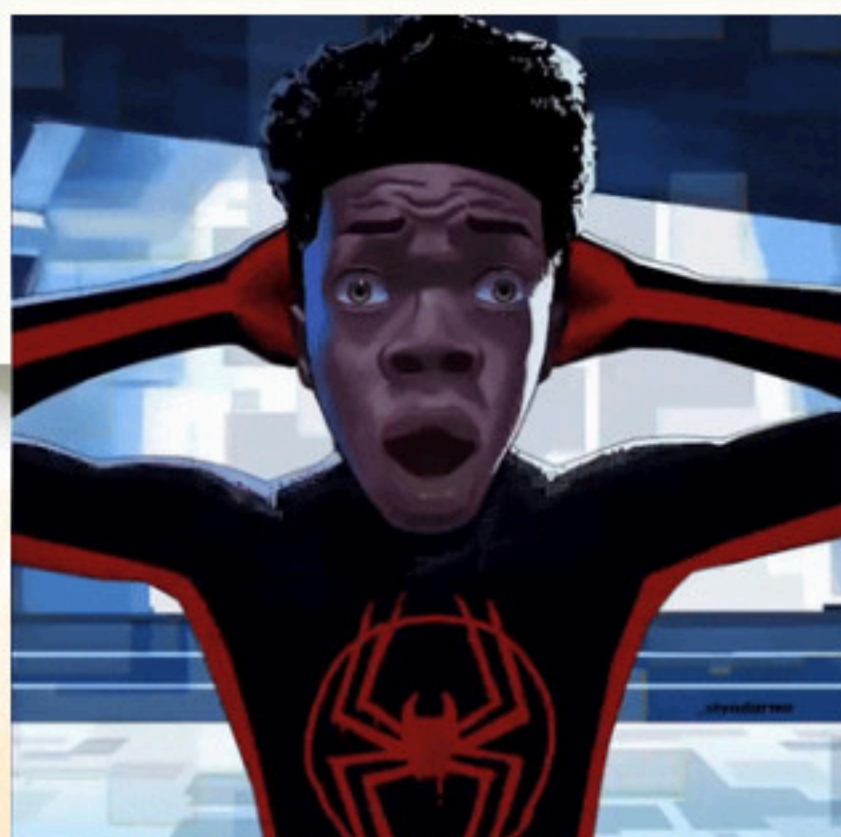
การลืมนำ Iterator ใน Rust เป็นแบบ Lazy Evaluation

Iterator ใน Rust มีคุณสมบัติ "Lazy" หมายความว่ามันจะไม่ประมวลผลอะไรเลย จนกว่าจะถูกเรียกใช้งานด้วย consuming adaptors (เช่น `.collect()`, `.sum()`, หรือถูกใช้งานใน `for` loop) มือใหม่มักเขียน chain ของ Iterator ทิ้งไว้โดยไม่ดึงค่าออกมา ทำให้โค้ดส่วนนั้นไม่ถูกทำงาน

Why??

Rust ออกแบบมาให้ทำงานเร็วและประหยัดทรัพยากร การเป็น Lazy ทำให้ Rust ไม่ต้องเสียเวลาสร้าง Collection ชั่วคราวในแต่ละขั้นตอนของ chain จนกว่าจะถึงจุดที่จำเป็นต้องใช้ผลลัพธ์จริงๆ

can you git gud?



incorrect code

```
fn main() {  
    let numbers = vec![1, 2, 3];  
  
    // โค้ดนี้จะไม่ทำอะไรเลย compiler จะแจ้งเตือน unused `Map`  
    numbers.iter().map(|x| println!("Processing: {}", x));  
}
```

correct code

```
fn main() {  
    let numbers = vec![1, 2, 3];  
  
    // วิธีที่ 1: ใช้ for loop เพื่อบริโภคค่า  
    for _ in numbers.iter().map(|x| println!("Processing: {}", x)) {  
        // Iterator ถูกกระตุ้นให้ทำงานแล้ว  
    }  
  
    // วิธีที่ 2: ใช้ .collect() หากต้องการนำผลลัพธ์ไปใช้ต่อ  
    let _processed: Vec<_> = numbers.iter().map(|x| x * 2).collect();  
}
```

Common Mistakes 2 ข้อผิดพลาดที่พบบ่อย

สับสนระหว่าง `iter()`, `iter_mut()`, และ `into_iter()`

การเลือกใช้เมธอดสร้าง Iterator ผิดประเภท ทำให้เกิดปัญหา Ownership โดยเฉพาะการเปลือยใช้ `into_iter()` ซึ่งจะย้ายกรรมสิทธิ์ (Move) ของตัวแปรไป ทำให้ไม่สามารถเรียกใช้ Collection ต่อกันได้อีก

Why??

- `.iter()`: ยืมค่าแบบอ่านอย่างเดียว (&T)
- `.iter_mut()`: ยืมค่าแบบแก้ไขได้ (&mut T)
- `.into_iter()`: ย้ายกรรมสิทธิ์แบบสมบูรณ์ (T) เหมาะสำหรับตอนที่ เราไม่ต้องการใช้ Collection ต่อกันอีกต่อไป

incorrect code

```
fn main() {  
    let names = vec![String::from("Alice"), String::from("Bob")];  
  
    // into_iter() จะกิน (consume) ownership ของ names เข้าไป  
    for name in names.into_iter() {  
        println!("Hello, {}", name);  
    }  
  
    // Error: borrow of moved value: `names`  
    println!("Total names: {}", names.len());  
}
```

correct code

```
fn main() {  
    let names = vec![String::from("Alice"), String::from("Bob")];  
  
    // ใช้ iter() เพื่อขอยืมค่า (Borrow) มาอ่านเท่านั้น  
    for name in names.iter() {  
        println!("Hello, {}", name);  
    }  
  
    // สามารถเรียกใช้ names ต่อได้ตามปกติ  
    println!("Total names: {}", names.len());  
}
```



Challenge

Ex1. การกิจส่งมาม่ากู้ภัยน้ำท่วม (mama rescue missions)

สถานการณ์น้ำท่วมปีนี้หนักหน่วงมาก เราจึงมี Vector ที่เก็บข้อมูลชื่อจังหวัดและระดับน้ำ (เซนติเมตร) ดังนี้

```
fn main() {
    let areas = vec![
        ("Chiang Rai", 150),
        ("Phayao", 80),
        ("Chiang Mai", 120),
        ("Bangkok", 15)
    ];

    let rescue_missions: Vec<String> = areas
        .iter()
        .filter(|&&(_, water_level)| water_level >= 100)
        .map(|&(province, _)| format!("{}", prepare boats and mama rn", province))
        .collect();

    println!("{:?}", rescue_missions);
}
```

คำถาม

เมื่อโปรแกรมรันเสร็จ ผลลัพธ์ (Output) ที่แสดงบนหน้าจอจะเป็นข้อความว่าจะอะไรบ้าง?

คำตอบ

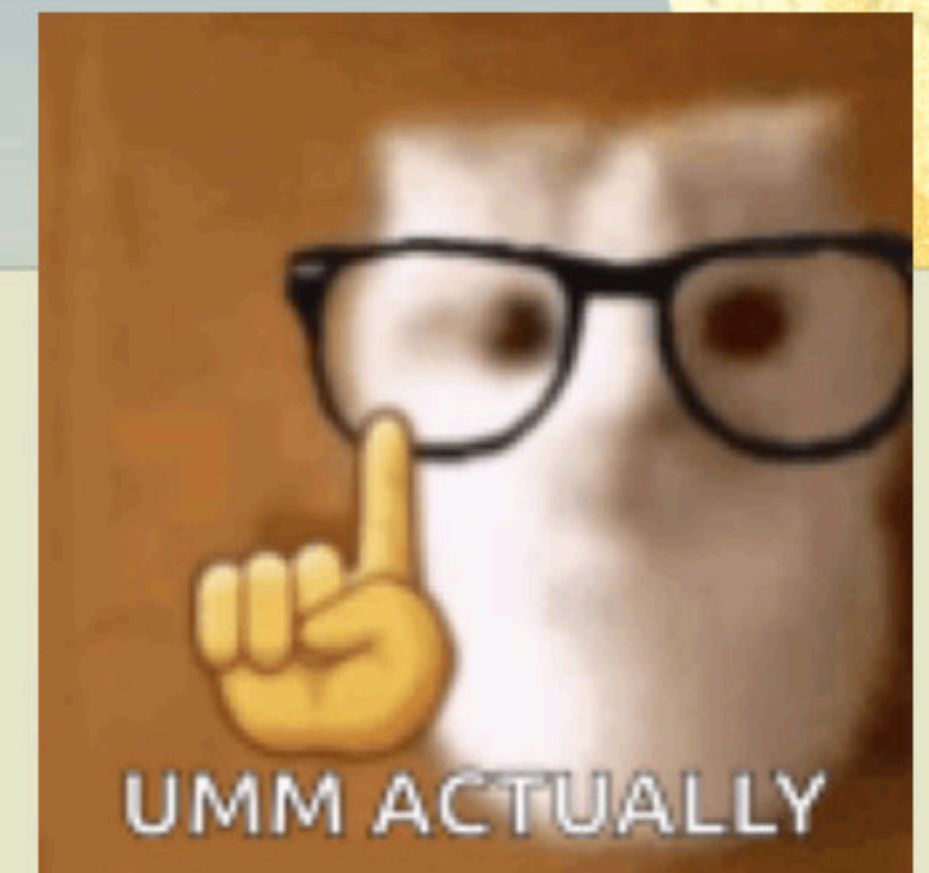
```
["Chiang Rai prepare boats and mama rn", "Chiang Mai prepare boats and mama rn"]
```

Explanation

โค้ดจะทำงานแบบ Lazy Evaluation โดยดึงข้อมูลมาทีละตัว:

1. Chiang Rai (150) -> ผ่าน filter -> เข้า map แปลงเป็นข้อความ
2. Phayao (80) -> ไม่ผ่าน filter (ถูกเตะทิ้งทันที)
3. Chiang Mai (120) -> ผ่าน filter -> เข้า map แปลงเป็นข้อความ
4. Bangkok (15) -> ไม่ผ่าน filter

สุดท้าย collect() จะรวบรวมข้อความที่ผ่านเข้ารอบทั้งหมดมาใส่กรอบเป็น Vector เดียวกัน



Challenge



Ex2. รวมเหรียญทองเอเชียนเกมส์ 2026 (total gold coin)

ในการแข่งขัน Aichi-Nagoya 2026 Asian Games ทีมชาติไทยคว้าเหรียญทองมาได้จากหลายกีฬา เรามี Vector เก็บชนิดกีฬาและจำนวนเหรียญทองที่ได้ ดังนี้

```
fn main() {  
    let medals = vec![  
        ("Sepak Takraw", 4),  
        ("Esports (RoV)", 1),  
        ("Tamiya", 5), //กีฬา Tamiya เพิ่งถูกบรรจุใหม่ที่สนาม Nagoya-grand prix  
        ("Muay Tha", 3)  
    ];  
  
    let total_gold = medals.iter().fold(2, |acc, &(_, count)| acc + count);  
  
    println!("Thailand total gold medals won: {} medals", total_gold);  
}
```

คำถาม

เมื่อโปรแกรมรันเสร็จ ผลลัพธ์ (Output) ที่แสดงบนหน้าจอจะเป็นข้อความว่าอะไรบ้าง?

คำตอบ

Thailand total gold medals won: 15 medals

Explanation

โจทย์ข้อนี้เป็นการทดสอบความเข้าใจพารามิเตอร์ของ fold(initial_value, closure)

- ใน Vector มีเหรียญรวมกันคือ $4 + 1 + 5 + 3 = 13$ เหรียญ
- แต่เราตั้งค่าเริ่มต้น (ตัวแปร acc รอบแรก) ไว้ที่ 2 (อาจจะเป็นเหรียญทองที่ยกยอดมาจากเมื่อวาน)
- ดังนั้นระบบจะนำ 2 เป็นตัวตั้ง แล้วค่อยๆ บวกเพิ่มเข้าไปที่ละรายการจนครบ ลัพธ์สุดท้ายที่ได้จึงเป็น $2 + 13 = 15$



Key Takeaways

สรุปสิ่งที่ควรรจำ

- Iterator is Lazy = ยังไม่ประมวลผลจนกว่าจะถูกใช้งาน
- `iter()` = ยืมข้อมูล ต้นฉบับสามารถนำไปใช้งานต่อได้
- `map()` = แปลงข้อมูลแต่ละตัว
- `filter()` = เก็บเฉพาะข้อมูลที่ต้องการ
- `fold()` = รวมข้อมูลทั้งหมดให้เหลือค่าเดียว และ Consume Iterator (กลืนแล้วเอาไปใช้ไม่ได้)
- `collect()` = สร้าง Collection ต้องรู้ชนิดปลายทาง เช่น `Vec<_>`
- Closure = ฟังก์ชันไม่ระบุชื่อ ใช้กับ `map`, `filter`, `fold` ได้
- Iterator Chain ได้ เช่น `iter → map → filter → collect`
- เกี่ยวข้องกับหัวข้อ `collection` โดยตรงเพราะ ต้องแปลงจาก `collection → iterator`

References

- **1) Rust Foundation. Iterator Trait — Rust Standard Library Documentation.**
<https://doc.rust-lang.org/std/iter/trait.Iterator.html>
- **2) Rust Foundation. Iterators — The Rust Programming Language.**
<https://doc.rust-lang.org/book/ch13-02-iterators.html>
- **3) Rust Foundation. Closures — The Rust Programming Language.**
<https://doc.rust-lang.org/book/ch13-01-closures.html>
- **4)GeeksforGeeks. Rust - Higher Order Functions.**
<https://www.geeksforgeeks.org/rust/rust-higher-order-functions/>
- **5)Crust of Rust: Iterators - Jon Gjengset.**
<https://www.youtube.com/watch?v=yozQ9C69pNs>
- **6)82: Iterators in Rust are Awesome - Rustfully.**
<https://www.youtube.com/watch?v=BEPhAlkw6x8>



Q&A

Group 20 - Iterators &
Higher-Order Programming